

Documentación del Proyecto — Pitahaya Biotec

Sitio Web de Gestión Biotecnológica para la Pitahaya Ecuatoriana

Índice

1. [Introducción](#)
 2. [Justificación Técnica](#)
 3. [Arquitectura del Sistema](#)
 4. [Base de Datos](#)
 5. [Roles y Permisos](#)
 6. [Flujo de Funcionamiento](#)
 7. [Módulos del Sistema](#)
 8. [Decisiones de Diseño](#)
 9. [Seguridad](#)
 10. [Despliegue y Operación](#)
-

1. Introducción

1.1 ¿Qué es Pitahaya Biotec?

Pitahaya Biotec es un **sitio web de gestión** diseñado para integrar y digitalizar los procesos de investigación, producción y comercialización de la pitahaya (*Hylocereus* spp.) en Ecuador. Es desarrollado para **ESPOCH** (Escuela Superior Politécnica de Chimborazo) e **INIAP** (Instituto Nacional de Investigaciones Agropecuarias).

1.2 Objetivo del Sistema

Centralizar en una sola plataforma los datos científicos, operativos y administrativos del proyecto pitahaya, permitiendo a productores, investigadores, técnicos y administradores acceder a información actualizada y tomar decisiones basadas en datos.

1.3 Problema que Resuelve

Antes del sistema, los datos de investigación, pasaportes FAO, inventario de germoplasma y solicitudes de semillas se manejaban en:

- Hojas de cálculo sueltas (Excel)
- Documentos de Word
- Cuadernos de campo físicos
- Comunicación por WhatsApp/email sin trazabilidad

Esto generaba:

- Duplicidad de datos
 - Pérdida de información
 - Imposibilidad de consultar datos históricos
 - Falta de estandarización (especialmente en datos FAO/MCPD)
 - Dificultad para generar reportes
-

2. Justificación Técnica

2.1 ¿Por qué un sitio web y no un gestor de contenido (CMS)?

Aspecto	Sitio Web a Medida (Pitahaya Biotec)	CMS (WordPress, Joomla, Drupal)
Personalización	Total. Cada módulo fue diseñado específicamente para el flujo de trabajo del proyecto pitahaya.	Limitado por los plugins y temas existentes.
Base de datos	Esquema relacional optimizado para datos científicos (FAO MCPD, germoplasma, investigaciones).	Estructura genérica (wp_posts, wp_meta) que fuerza datos científicos a tablas EAV ineficientes.
Roles	Sistema de roles granular con 4 niveles y permisos específicos por módulo.	Roles genéricos (admin, editor, autor, suscriptor) difíciles de adaptar.
API	API REST completa (63+ endpoints) que permite integraciones futuras (apps móviles, IoT).	Las APIs de CMS son genéricas y no exponen lógica de negocio.
Rendimiento	Liviano, sin base de datos embebida ni capas de abstracción innecesarias.	Los CMS añaden sobrecarga (temas, plugins, caché, consultas extra).
Mapa interactivo	Leaflet.js integrado con datos de la misma BD y CRUD embebido.	Requiere plugins de terceros con limitaciones y costos.
Calculadora científica	Fertiliriego con lógica de negocio específica (etapas fenológicas, tipos de suelo).	Imposible sin desarrollo a medida.
Mantenimiento	Un solo archivo server.js + HTML estáticos. Sin actualizaciones forzadas.	Actualizaciones constantes de core, plugins y temas; riesgo de incompatibilidad.

Conclusión: Un CMS es adecuado para blogs o tiendas genéricas, pero para un sistema con lógica científica, datos estructurados complejos y flujos de trabajo específicos, un desarrollo a medida es la opción correcta.

2.2 ¿Por qué Node.js + Express?

- **Rendimiento:** Asíncrono y orientado a eventos, ideal para endpoints de API con múltiples consultas a BD.
- **Ligereza:** Sin capas de framework pesadas. Express es minimalista.
- **JWT nativo:** jsonwebtoken se integra naturalmente.
- **Despliegue en Vercel:** Serverless Node.js sin configuración de servidor.
- **Ecosistema npm:** Acceso a paquetes como @supabase/supabase-js, bcryptjs, cors.

2.3 ¿Por qué Supabase (PostgreSQL)?

- **SQL real:** Consultas complejas con JOINS, agregaciones, funciones de ventana. Firebase (NoSQL) no lo permite.
- **Relaciones:** Los datos del proyecto son inherentemente relacionales (usuarios → pedidos → productos).
- **Escalabilidad:** PostgreSQL maneja desde prototipos hasta producción con millones de registros.
- **Supabase específicamente:** Añade Storage, API REST automática y panel de administración web.
- **Costo cero inicial:** Tiene un generoso plan gratuito.

2.4 ¿Por qué frontend vanilla (sin React, Vue, etc.)?

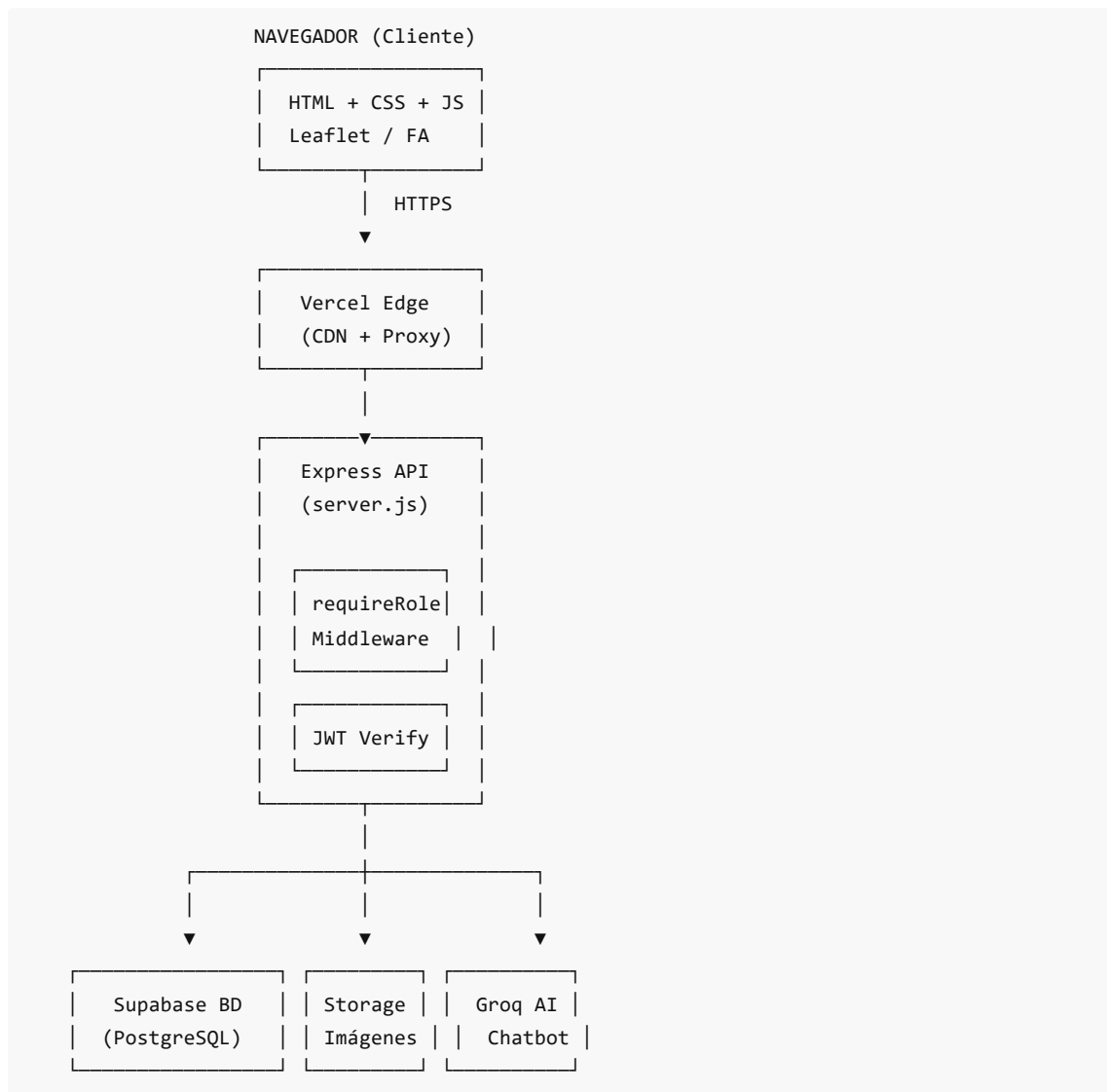
- **Simplicidad:** El equipo no necesita aprender un framework moderno. HTML + CSS + JS es universal.
- **Rendimiento:** Sin compilación, sin bundle, sin virtual DOM innecesario para una app de gestión.
- **Tamaño:** Cada página es un HTML independiente que solo carga lo necesario.
- **Mantenimiento:** Cualquier persona con conocimientos básicos de web puede hacer cambios.

2.5 ¿Por qué Vercel?

- **Despliegue automático:** Conectar con GitHub + `git push` = producción actualizada.
- **Serverless:** No hay que administrar servidores, parches de seguridad, ni escalar.
- **CDN global:** Los archivos estáticos se sirven desde el edge.
- **Capa gratuita generosa:** Suficiente para producción académica.

3. Arquitectura del Sistema

3.1 Diagrama de Arquitectura



3.2 Flujo de una Solicitud

1. El usuario hace clic en un botón (ej: "Guardar Accesoión FAO")
2. El interceptor `nav.js` agrega automáticamente el JWT al fetch
3. La solicitud llega a Vercel → Express
4. `requireRole` verifica el JWT y el rol del usuario
5. El endpoint procesa la solicitud contra Supabase
6. La respuesta regresa al frontend
7. El frontend actualiza la interfaz

3.3 Componentes del Sistema

Backend (server.js):

- Servidor Express con 63 endpoints REST
- Middleware de autenticación JWT + bcrypt
- Middleware de autorización por roles
- Log de actividad (tabla logs_actividad)
- Chatbot con conocimiento local (19+ temas sobre pitahaya)
- Exportación de datos (CSV, JSON, TXT)
- SSE (Server-Sent Events) para noticias en tiempo real

Frontend (Public/):

- 22 páginas HTML
- 4 archivos CSS (tokens, estilos globales, dashboards)
- 3 archivos JS (login, navegación, dashboard)
- Mapas interactivos con Leaflet.js
- Chatbot flotante con IA
- Diseño responsive (adaptable a móviles)

4. Base de Datos

4.1 Modelo Entidad-Relación

```
usuarios (1) — (N) noticias
usuarios (1) — (N) likes
usuarios (1) — (N) comentarios
usuarios (1) — (N) semillas
usuarios (1) — (N) pedidos
usuarios (1) — (N) soporte_mensajes
usuarios (1) — (1) perfiles_investigador
usuarios (1) — (N) logs_actividad

noticias (1) — (N) likes
noticias (1) — (N) comentarios

perfiles_investigador (1) — (N) investigaciones
```

4.2 Diccionario de Tablas

Tabla	Propósito	Registros Esperados
-------	-----------	---------------------

usuarios	Cuentas de usuario con rol y contraseña hasheada	10-100
noticias	Artículos de novedades con imágenes múltiples	50-500
likes	Likes de noticias (par único usuario+noticia)	100-5000
comentarios	Comentarios en noticias	100-2000
variedades	Catálogo de variedades de pitahaya	10-50
fao_passport	Acciones de germoplasma estándar MCPD	100-10000
publicaciones	Artículos científicos y tesis	20-200
soporte_mensajes	Tickets de soporte técnico	50-500
logs_actividad	Auditoría de todas las operaciones CRUD	Ilimitado
semillas	Solicitudes de semillas de productores	50-500
pedidos	Pedidos de bioproductos	50-500
bioproductos	Catálogo de insumos biotecnológicos	10-100
puntos_monitoreo	Puntos georreferenciados de campo	50-1000
perfiles_investigador	Perfiles profesionales extendidos	5-50
investigaciones	Proyectos de investigación asociados	10-100

4.3 Justificación del Esquema

Por qué tablas separadas y no una tabla genérica:

- Cada entidad tiene atributos distintos (ej: FAO passport tiene latitud, altitud, estado biológico; variedades tiene beneficios, producción, características)
- Las consultas son más rápidas y claras (SELECT * FROM fao_passport WHERE pais = 'Ecuador')
- Las restricciones CHECK y NOT NULL se aplican por columna
- Las FK mantienen integridad referencial
- Escalabilidad: cada tabla crece independientemente

Por qué no usar una base de datos NoSQL (MongoDB, Firebase):

- Los datos son inherentemente relacionales (un pedido pertenece a un usuario, tiene productos, tiene estados)
- Las consultas de agregación (likes por noticia, pedidos por usuario) son naturales en SQL
- El estándar FAO MCPD es tabular por naturaleza
- PostgreSQL permite consultas geoespaciales (PostGIS) para el módulo de mapas

4.4 Políticas de Seguridad (RLS)

No se implementaron Row Level Security en Supabase porque:

- Toda la lógica de autorización se maneja en el middleware de Express
 - El service_key se usa en el servidor, no se expone al cliente
 - Es más fácil depurar y auditar en una sola capa
-

5. Roles y Permisos

5.1 Definición de Roles

Rol	Descripción	Ejemplo de Usuario
admin	Administrador del sistema. Control total.	Coordinador del proyecto
investigador	Científico que genera y gestiona datos de investigación.	Investigador ESPOCH/INIAP
tecnico	Técnico de campo que opera en terreno.	Técnico agrícola
productor	Agricultor que solicita insumos y consulta información.	Productor de pitahaya

5.2 Matriz de Permisos

Módulo	Leer	Crear	Editar	Eliminar
Noticias	Todos	admin, investigador, tecnico	admin, investigador, tecnico	admin, investigador, tecnico
Comentarios	Todos	Todos	—	admin, investigador, tecnico
FAO Passport	Todos	admin, investigador, tecnico	admin, investigador, tecnico	admin, investigador, tecnico
Variedades	Todos	admin, investigador, tecnico	admin, investigador, tecnico	admin, investigador, tecnico
Publicaciones	Todos	admin, investigador, tecnico	admin, investigador, tecnico	admin, investigador, tecnico
Bioproductos	Todos	admin, investigador, tecnico	admin, investigador, tecnico	admin, investigador, tecnico
Mapa (puntos)	Todos	admin, investigador, tecnico	admin, investigador, tecnico	admin, investigador, tecnico
Semillas (solicitar)	Todos	Todos	—	—
Pedidos (crear)	Propios	Todos	—	—
Soporte (enviar)	Propios	Todos	—	—
Soporte (responder)	Todos	admin, tecnico	—	—

Usuarios (gestionar)	admin	admin	admin	admin
Perfil profesional	admin, investigador	investigador	investigador	investigador
Panel Admin	admin	—	—	—

5.3 Implementación de Permisos

Los permisos se implementan en dos capas:

1. Backend (middleware requireRole):

```
app.post('/api/fao-passport', requireRole(['admin', 'investigador', 'tecnico']), async (req, res) => {
  // Solo admin, investigador y tecnico pueden crear accesiones FAO
});
```

2. Frontend (ocultar botones):

```
const puedeEditar = (userRole === 'admin' || userRole === 'investigador' || userRole === 'tecnico');
// Si puedeEditar es false, los botones de editar/eliminar no se renderizan
```

5.4 ¿Por qué estos roles?

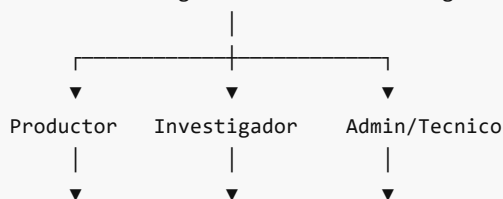
Los roles fueron definidos en conjunto con el equipo de ESPOCH/INIAP para reflejar la estructura organizativa real:

- **Productor:** Es el usuario final que se beneficia del proyecto. Necesita acceder a información (variedades, noticias) y hacer solicitudes (semillas, pedidos, soporte). No necesita modificar datos científicos.
- **Investigador:** Genera los datos científicos. Publica noticias, registra variedades, ingresa datos FAO, escribe publicaciones. No necesita administrar usuarios ni responder soporte genérico.
- **Técnico:** Opera en campo. Agrega puntos al mapa, registra bioproductos, y puede responder soporte porque es el primer contacto con productores.
- **Admin:** Coordina el proyecto. Gestiona usuarios, asigna roles, y tiene acceso completo para auditoría.

6. Flujo de Funcionamiento

6.1 Ciclo de Vida de un Usuario

Registro → Login → JWT → Navegación → Acciones → Logout



Solicitar semillas	Crear datos científicos	Gestionar sistema
Pedidos	Publicar noticias	Responder soporte
Comentar		

6.2 Flujo de Datos Científicos (FAO Passport)

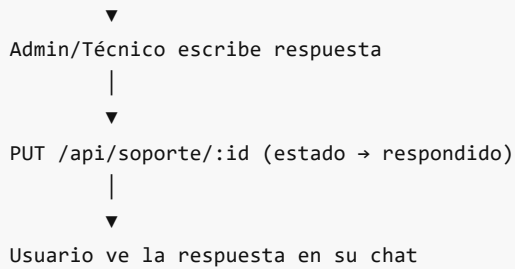
```
graph TD; A[Recolección de campo (GPS, fotos, notas)] --> B[Ingreso en sistema (formulario 4 pestañas)]; B --> C[Validación (middleware requireRole)]; C --> D[Almacenamiento en Supabase (tabla fao_passport)]; D --> E[Disponible para consulta (tabla, detalle, exportación)]; E --> F[Vinculación con mapa (coordenadas en Leaflet)];
```

6.3 Flujo de Solicitud de Semillas

```
graph TD; A[Productor selecciona variedad y cantidad] --> B[Formulario → POST /api/semillas]; B --> C[Admin ve solicitud en panel Admin]; C --> D[Admin cambia estado (pendiente → aceptado/rechazado)]; D --> E[Productor ve el estado actualizado];
```

6.4 Flujo de Soporte Técnico

```
graph TD; A[Usuario escribe mensaje en chat de soporte] --> B[POST /api/soporte → tabla soporte_mensajes]; B --> C[Admin/Técnico ve en Bandeja de Soporte];
```

7. Módulos del Sistema

7.1 Autenticación (Login/Registro)

Cómo funciona:

- Registro: POST /api/register → bcrypt hash + JWT
- Login: POST /api/login → verificar password → JWT
- JWT almacenado en localStorage → enviado automáticamente por nav.js

Medidas de seguridad:

- Contraseñas hasheadas con bcrypt (salt rounds = 10)
- JWT con expiración de 7 días
- Fallback a comparación plana para migración de usuarios legacy
- No se almacenan tokens en cookies (XSS protection vía localStorage + HttpOnly no disponible pero mitigado por el interceptor)

7.2 Dashboard General (Inicio)

Tarjetas de acceso rápido a todos los módulos del sistema. Diseñado como landing page post-login. Cada tarjeta tiene un icono, título y descripción breve.

7.3 Atlas de la Pitahaya (Dashboard Biotec)

Tablero estratégico con:

- Estadísticas de producción nacional
- Gráficos de consumo (nacional e internacional)
- Datos de exportación
- Inventario dinámico FAO (consulta en tiempo real)
- Laboratorio virtual de clasificación por peso

7.4 Noticias

Sistema completo de publicación con:

- Múltiples imágenes por noticia (hasta 10, almacenadas en Supabase Storage)
- Likes con toggle (par único usuario+noticia)
- Comentarios en cada noticia
- SSE (Server-Sent Events) para actualización en tiempo real
- Botones de compartir (WhatsApp, Facebook)

7.5 FAO Passport

Implementación del estándar **MCPD** (Multi-Crop Passport Descriptors) de FAO/Bioversity International. Los 16 campos cubren:

- Identificación (código, acceso, colecta, nombre accesión)
- Taxonomía (género, especie, subtaxa, nombre común)
- Origen (país, ubicación, coordenadas, altitud, fecha)
- Colecta (estado biológico, fuente de colecta)

El formulario está organizado en 4 pestañas para facilitar el ingreso.

7.6 Variedades (Tipos)

Catálogo de variedades de pitahaya con:

- Nombre común y científico
- Imagen por variedad
- Beneficios, localidad, producción, características
- Modal de detalle completo

7.7 Mapa de Cultivos

Integración con **Leaflet.js** para visualización geoespacial:

- Marcadores coloreados por estado de salud
- Popups con información de cada punto
- Filtros por variedad y estado
- Buscador de ubicaciones (Nominatim)
- CRUD completo con clic en mapa para agregar coordenadas

7.8 Fertirriego

Calculadora agronómica interactiva:

- Litros de agua recomendados por planta/día según edad, suelo y temperatura
- Tabla nutricional por etapa fenológica (crecimiento, floración, fructificación, post-cosecha)
- Rangos de NPK y micronutrientes por etapa

7.9 Bioproductos y Pedidos

- Catálogo de bioproductos con precio e imagen
- Sistema de pedidos con estados (pendiente → aceptado → enviado)
- Validación de stock básica

7.10 Publicaciones Científicas

Repositorio de artículos y tesis con:

- DOI y enlace a PDF
- Filtro por tipo (artículo/tesis)
- Búsqueda por título, autor, revista

7.11 Investigadores

Directorio público con:

- Perfiles profesionales (foto, biografía, especialidad, institución)
- Enlaces académicos (ORCID, Google Scholar, LinkedIn)
- Proyectos de investigación vinculados

- Búsqueda y filtros

7.12 Soporte Técnico

Sistema de tickets con:

- Chat de usuario (envía mensaje, ve respuestas)
- Bandeja de soporte (admin/técnico ve todos los tickets y responde)
- Estados (pendiente → respondido)
- Contacto directo (teléfono, email, horario)

7.13 Chatbot IA

Asistente virtual con:

- Integración opcional a Groq API (Llama 3.3-70B)
- Base de conocimiento local con 19+ temas (cultivo, exportación, nutrición, plagas, etc.)
- Respuestas inmediatas sin depender de API externa
- Interfaz flotante accesible desde cualquier página

7.14 Panel Admin

Gestión centralizada para administradores:

- Usuarios: cambiar roles, eliminar cuentas
 - Semillas: gestionar solicitudes
 - Pedidos: actualizar estados
 - Soporte: bandeja de tickets
-

8. Decisiones de Diseño

8.1 Interfaz de Usuario

Paleta de colores:

- Verde principal (#167538): Representa el campo, la agricultura y lo biotecnológico
- Amarillo acento (#F1D400): Representa la pitahaya (pulpa amarilla) y la energía
- Fondo claro (#F3F7FB): Máxima legibilidad
- Texto oscuro (#121B32): Alto contraste

Tipografía:

- Georgia (serif): Elegida por el usuario por su legibilidad en pantalla
- Navegación en sans-serif (Segoe UI): Para contraste visual

Diseño responsive:

- Mobile-first con media queries en 768px y 480px
- Menú hamburguesa para móviles
- Tablas con scroll horizontal en pantallas pequeñas

8.2 Por qué tabs en formularios largos

El formulario FAO Passport tiene 16 campos. En lugar de un scroll infinito, se organizó en 4 pestañas (Identificación, Taxonomía, Origen, Colecta) para:

- Reducir la carga cognitiva (el usuario solo ve 4-5 campos a la vez)

- Agrupar campos relacionados semánticamente
- Permitir validación por sección

8.3 Por qué fetch interceptor en nav.js

En lugar de modificar cada llamada fetch en las 22 páginas, se parchea globalmente `window.fetch` para:

- Incluir automáticamente el JWT en todas las llamadas a `/api/`
- Caer grácilmente a `x-usuario-id` si no hay JWT (migración legacy)
- Mantener la lógica de autenticación en un solo lugar

8.4 Por qué no Single Page Application (SPA)

Cada página es un HTML independiente. Ventajas:

- Carga inicial rápida (solo lo necesario para esa página)
- Funciona sin JavaScript (hasta cierto punto)
- SEO natural (Google indexa cada página)
- Simplicidad de desarrollo y depuración
- Sin framework frontend que aprender

Desventaja asumida: La navegación recarga la página, pero dado que es una herramienta de gestión (no una red social), es aceptable.

8.5 Por qué el chatbot no requiere API key

La base de conocimiento local cubre 19+ temas con respuestas predefinidas, lo que significa:

- Funciona sin conexión a internet (después de cargar la página)
- Sin costos de API
- Respuestas inmediatas
- Fácil de ampliar (solo agregar entradas al array CONOCIMIENTO)

Opcionalmente se puede conectar a Groq API para respuestas generadas por IA.

9. Seguridad

9.1 Autenticación

- **JWT (JSON Web Tokens):** Firmados con HMAC-SHA256, expiración de 7 días
- **bcryptjs:** Contraseñas hasheadas con 10 rounds de sal
- **JWT_SECRET:** Variable de entorno, con fallback local para desarrollo

9.2 Autorización

- **Middleware requireRole:** Verifica JWT + rol antes de cada operación CRUD
- **Roles en frontend:** Botones de acción se ocultan según el rol
- **Fallback legacy:** Usuarios sin JWT pueden operar mediante `x-usuario-id`

9.3 Protección de Datos

- Service key de Supabase solo en servidor (nunca expuesta al cliente)
- Anon key de Supabase con RLS desactivado (control en servidor)
- Sin contraseñas en localStorage (solo JWT y datos básicos)
- Logs de actividad para auditoría

9.4 Validación

- Server-side: Validación de campos requeridos en todos los endpoints
- Frontend: Validaciones básicas antes de enviar
- SQL Injection: Prevenido por el cliente Supabase (parametriza automáticamente)

9.5 Limitaciones Conocidas

- No hay rate limiting (podría agregarse con express-rate-limit)
- No hay HTTPS forzado (lo maneja Vercel automáticamente)
- No hay recuperación de contraseña (pendiente)
- No hay expiración de sesión forzada (solo por expiración del JWT a 7 días)

10. Despliegue y Operación

10.1 Ciclo de Desarrollo

1. Desarrollo local (npm run dev)
2. Pruebas en localhost:3000
3. Commit a main en GitHub
4. Push automático → Vercel despliega
5. Producción en <https://pitahaya-project.vercel.app>

10.2 Requisitos de Operación

- **Frontend:** Cualquier navegador moderno (Chrome, Firefox, Edge, Safari)
- **Backend:** Node.js 22.x (Vercel serverless)
- **BD:** Supabase PostgreSQL 15.x (plan gratuito: 500 MB)
- **Storage:** Supabase Storage (plan gratuito: 1 GB)

10.3 Monitoreo

- **Logs:** Vercel Dashboard (deployment logs)
- **Base de datos:** Supabase Dashboard (consultas, rendimiento, tamaño)
- **Auditoría:** Tabla logs_actividad con todas las operaciones CRUD

10.4 Respaldos

- Supabase realiza backups automáticos (retención según plan)
- El código fuente está en GitHub
- Las imágenes están en Supabase Storage

10.5 Escalabilidad

El sistema actual soporta:

- **Usuarios:** Cientos (la autenticación no tiene cuello de botella)
- **Registros:** Decenas de miles por tabla (PostgreSQL lo maneja sin índices adicionales)
- **Imágenes:** Hasta 1 GB en Storage (plan gratuito)
- **Peticiones:** Las que permita el plan gratuito de Vercel (100k solicitudes/mes)

Para escalar más:

- Agregar índices en columnas de búsqueda frecuente
 - Migrar a plan de pago de Vercel y Supabase
 - Implementar caché con Redis (si fuera necesario)
-

Apéndice A: Glosario

Término	Significado
MCPD	Multi-Crop Passport Descriptors — estándar FAO para datos de germoplasma
FAO	Food and Agriculture Organization of the United Nations
Germoplasma	Material genético que se conserva para futura investigación y mejora
Accesión	Muestra única de germoplasma registrada en un banco
Bioteconología	Uso de organismos vivos para desarrollar productos útiles
Fenología	Estudio de las etapas del desarrollo de las plantas
Fertirriego	Técnica de aplicar fertilizantes disueltos en el agua de riego
Leaflet	Biblioteca JavaScript de mapas interactivos de código abierto
JWT	JSON Web Token — estándar para transmitir información de autenticación
bcrypt	Algoritmo de hashing de contraseñas con sal incorporada

Apéndice B: Datos del Proyecto

- **URL:** <https://pitahaya-project.vercel.app>
- **Repositorio:** <https://github.com/Karen-404/PitahayaProject>
- **Base de datos:** Supabase PostgreSQL
- **Framework backend:** Express.js sobre Node.js
- **Frontend:** HTML + CSS + JavaScript vanilla
- **Hosting:** Vercel (serverless)
- **Equipo:** ESPOCH / INIAP
- **Año:** 2026

Documento generado el 12 de julio de 2026.